SIMATS ENGINEERING

CSA17 – ARTIFICIAL INTELLIGENCE

ASSIGNMENT 1

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.

Student Name	**T.Jaya Krishna Reddy**	**Register Number**	**192572036**
Department / Section	**B.Tech CSE AI**	**Academic Year / Semester**	**2025-2026**
Faculty Name	**Dr. Krishna Kumar Kathiresan**	**Nature of Assignment**	**Individual**
Duration	**1 Week**	**Total Marks**	**50**
Date of Submission	**2-11-2026**	**Date of Evaluation**	

MARK SHEET (Rubric-based Evaluation)

	Task Title	Max	Marks Awarded	Remarks
1	Design the Agent and the Search Algorithms	5		
2	Identify the Basic Operation	2		
3	Derive Time and Space Complexity Functions	6		
4	Best, Average and Worst-Case Analysis	4		
5	Asymptotic Analysis	3		
6	Formulate the Recurrence Relation	3		
7	Solve the Recurrence Relation	4		
8	Implement and Test the Algorithms	4		
9	Record Experimental Performance	3		
10	Compare Theory with Experiments	4		
11	Recommend and Defend the Final Design	4		
12	Re-evaluate and Adapt the Design (Day 4)	8		
	TOTAL	**50**		

OVERALL REMARKS:

mohanram		
Student Signature	**Faculty Signature**	**Program Director**

ASSIGNMENT 1 – CO1

ARTIFICIAL INTELLIGENCE – CSA17

CO1: Foundation and history of AI, intelligent agents (agents and environments, rationality, nature of environments, structure of agents), problem-solving agents, problem formulation, searching for solutions, uninformed search strategies and search with partial information.

Assignment Title: Design, Analyse and Validate an Efficient Search Strategy for a Hospital Medicine-Delivery Robot **(Total Marks – 50)**

Duration: 1 Week

Scenario — Hospital Medicine-Delivery Robot

A multi-block hospital uses an autonomous robot to carry medicines from the central pharmacy to wards. The building is modelled as a graph of locations (junctions, corridors, lifts, ward doors). The robot may move between connected locations and every move has a travel-time cost.

As new wings are added, the state space of the robot may grow:

1,000 → 10,000 → 100,000 → 1,000,000+ states

You are required to investigate, design and compare uninformed search strategies by which the robot finds a route from the pharmacy to a requested ward as the hospital grows.

Your solution must remain practical for both small and very large state spaces. Any preprocessing (for example, building adjacency lists or a duplicate-detection structure) must be included in your efficiency analysis.

Design Challenge (Day 4): The robot's localisation sensor now fails intermittently, so the robot no longer knows exactly where it is. Re-evaluate your original design and decide whether it should be retained, modified or redesigned.

TASK 1 – Design the Agent and the Search Algorithms (5 Marks)

Before designing algorithms, define the agent and the problem:

- Specify the PEAS description of the robot and classify its environment (observable, deterministic, episodic/sequential, static/dynamic, discrete/continuous, single/multi-agent), justifying each choice
- State which view of AI (acting rationally, thinking rationally, acting humanly, thinking humanly) the robot follows and why it is a rational problem-solving agent. Relate the problem type briefly to the early history of AI problem solving
- Formulate the problem: states, initial state, actions, transition model, goal test, path cost

Then design and propose at least **three meaningfully different** uninformed search strategies for finding a route. Iterative and recursive implementations of the same underlying strategy will not be considered two different approaches.

Your proposed approaches must include:

- At least one iterative approach (queue- or stack-based)
- At least one recursive approach
- A clear explanation of the basic idea behind each approach

- Pseudocode or an appropriate flowchart for each approach
- A brief justification for selecting each approach for investigation
- Any preprocessing required, such as adjacency-list construction or an explored set

You are expected to select the approaches independently. The appropriateness and diversity of the selected strategies are part of the assessment.

Evidence Expected: PEAS and environment classification, complete problem formulation, three distinct strategies with pseudocode/flowcharts, preprocessing requirements and justification for each.

TASK 2 – Identify the Basic Operation (2 Marks)

For each proposed algorithm, identify the basic operation that predominantly determines its execution time.

Discuss:

- What the basic operation is (for example, node expansion, successor generation, goal test or frontier insertion)
- Where it occurs in the algorithm
- Why it is appropriate for efficiency analysis
- How the number of times it executes may change as the state-space size increases

Do not simply name an operation. Relate it to the behaviour of the algorithm.

Evidence Expected: Correct identification and justification of the basic operation for each algorithm.

TASK 3 – Derive the Time- and Space-Complexity Functions Mathematically (6 Marks)

Derive the time and space complexity for each proposed approach using mathematical analysis.

Discuss:

- The input size (branching factor b, shallowest-goal depth d, maximum depth m, optimal cost C^*, minimum step cost ε)
- The number of times the basic operation executes, derived by summing the nodes at each level
- The mathematical expression for time and for memory (frontier size)
- Any preprocessing cost involved
- The overall cost when preprocessing and searching are considered together

You must show the mathematical reasoning. Merely stating that an algorithm is O(b^d) will not receive full credit.

Evidence Expected: A clear derivation of time and space functions for each approach, including operation counts and preprocessing costs.

TASK 4 – Analyse Best, Average and Worst-Case Behaviour (4 Marks)

Analyse each algorithm under different input conditions.

Discuss, wherever applicable:

- What input condition produces the best case (for example, the goal lies on the first branch explored)
- What condition produces the average case
- What condition produces the worst case (for example, goal at maximum depth, goal unreachable, cheap-edge detours)

- How the number of nodes generated changes in each case, and why

Do not simply provide a table of complexity values. Explain the conditions that lead to each case.

Evidence Expected: A justified best-, average- and worst-case analysis supported by mathematical reasoning.

TASK 5 – Perform Asymptotic Analysis (3 Marks)

Express and compare the growth behaviour of the algorithms using O, Ω and/or Θ.

Discuss:

- How each algorithm behaves as the state space becomes very large
- How the growth rates of the approaches differ in time and in memory
- What the growth means when the hospital grows from 1,000 to 1,000,000+ states, for example, whether memory or time becomes the limiting factor first

Interpret the behaviour in the context of the hospital robot rather than merely stating the notation.

Evidence Expected: Correct asymptotic representation and a meaningful comparison of growth behaviour.

TASK 6 – Formulate the Recurrence Relation (3 Marks)

For the recursive approach, formulate a recurrence relation directly from the behaviour of your algorithm.

Identify clearly:

- The size of the original problem (depth limit or remaining depth)
- The size of the recursive subproblem
- The number of recursive calls (related to the branching factor)
- The work performed outside the recursive calls
- The base case
- How these components lead to the recurrence

The recurrence must be derived from your algorithm, not copied from a textbook.

Evidence Expected: A correct recurrence with clear identification of its components and base case.

TASK 7 – Solve the Recurrence Relation (4 Marks)

Solve the recurrence from Task 6 using an appropriate analytical method (substitution, iteration, recursion tree or another method).

Discuss:

- Why the selected method is applicable
- The steps involved in solving the recurrence
- The resulting complexity
- What the result indicates about the recursive algorithm as depth or branching factor increases
- How the result extends to iterative deepening, where the recursive search is repeated for increasing limits

Show the analytical process, not only the final result.

Evidence Expected: A complete solution with justification of the method and interpretation of the

result.

TASK 8 – Implement and Test the Algorithms (4 Marks)

Implement all proposed algorithms in Python and test them experimentally.

Use generated hospital graphs of approximately:

1,000 → 10,000 → 100,000 → 1,000,000 states

Ensure that:

- All algorithms return correct routes
- The same or equivalent graphs and start/goal pairs are used for comparison
- Testing conditions are consistent and the random seeds are reported
- Test cases include a reachable goal, an unreachable goal and a start equal to the goal
- The implementation corresponds to the algorithms analysed in Tasks 1–7

The purpose of implementation is to generate evidence for validating your theoretical analysis.

Evidence Expected: Working code, test inputs, expected behaviour and actual outputs, with a reproducible testing procedure.

TASK 9 – Record Experimental Performance (3 Marks)

Collect and present performance data for each algorithm across the different state-space sizes.

Record, with code and outputs as proof:

- Nodes generated and nodes expanded
- Maximum frontier size (memory proxy)
- Solution depth and path cost, and whether the path is optimal
- Execution time (averaged over at least 5 runs)

Present the results using tables and graphs (use a log scale where appropriate) so that approaches can be compared easily.

Evidence Expected: Systematically collected experimental data with clear tables and graphs.

TASK 10 – Compare Theory with Experimental Behaviour (4 Marks)

Compare your theoretical analysis with the experimental results.

Discuss:

- Whether the observed growth of nodes generated and of frontier size agrees with the theoretical complexity
- Which algorithm scales better as the state space increases
- Any differences between theory and observation, and the possible reasons (graph structure, duplicates, tie-breaking, language overhead)

Interpret the results rather than simply reproducing tables or graphs.

Evidence Expected: A reasoned comparison between theoretical and observed performance.

TASK 11 – Recommend and Defend the Final Design (4 Marks)

Based on your mathematical analysis and experimental findings, recommend the most appropriate search strategy for the hospital robot.

Discuss:

- Which approach you recommend and why it is more appropriate than the alternatives
- Its theoretical efficiency and experimentally observed performance
- Any preprocessing cost, trade-offs or limitations
- Whether it remains practical as the hospital grows

Your recommendation must be based on your own evidence. Simply choosing the algorithm with the lowest Big-O complexity is not sufficient.

Evidence Expected: A technically justified recommendation supported by analysis and experiment.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design (8 Marks)

Changed Requirement

After a power restart or sensor fault, the robot no longer knows exactly where it is. It knows only that it is at one of *k* possible junctions. It must still reach the pharmacy docking station, whichever junction it is actually at.

Re-evaluate your original design. You may **RETAIN → MODIFY → or REDESIGN** your original solution.

A. Impact Analysis – 2 Marks

- What has changed from the original problem and which original assumptions (fully observable, known start state) are affected?
- Why does the robot's knowledge now form part of the state?

B. Evaluation of the Original Design – 2 Marks

- Can your original algorithm still be applied? What does a "state" become?
- How large can the new search space be for *k* possible locations, and is the original design acceptable?

C. Adaptation or Redesign – 2 Marks

- Describe the revised strategy (for example, search over belief states, or search with local sensing and a conditional plan), what has changed and the effect on complexity
- If you retain the original solution, give sufficient technical justification

D. Evidence-Based Defence – 2 Marks

- Defend your decision using mathematical analysis and a small experiment (belief states generated and plan length for *k* = 2, 4, 8, with and without a local sensor), with code and output as proof

MARK DISTRIBUTION – ASSIGNMENT 1

Task	Marks
Task 1 – Agent and Algorithm Design	5
Task 2 – Basic Operation	2

Task	Marks
Task 3 – Mathematical Complexity Analysis	6
Task 4 – Best/Average/Worst-Case Analysis	4
Task 5 – Asymptotic Analysis	3
Task 6 – Recurrence Formulation	3
Task 7 – Recurrence Solution	4
Task 8 – Implementation & Testing	4
Task 9 – Experimental Performance	3
Task 10 – Theory vs Experiment	4
Task 11 – Final Design Recommendation	4
Subtotal	**42**
Task 12 – Day-4 Design Challenge	8
TOTAL	**50**

Name : T.Jaya Krishna Reddy

Reg.No. :192572036

Course: Artificial Intelligence

Slot D

Assignment 1: Design, Analyse and Validate an Efficient Search Strategy for a Hospital Medicine-Delivery Robot

TASK 1 – Design the Agent and the Search Algorithms (5 Marks)

1.1 PEAS Description

PEAS stands for Performance Measure, Environment, Actuators and Sensors.

Component	Description
Performance Measure	Deliver the correct medicine to the correct ward, minimize travel time, avoid collisions, maintain safety and complete delivery reliably.
Environment	Hospital corridors, junctions, lifts, pharmacy, ward doors, patients, staff and obstacles.
Actuators	Wheels, motors, steering system, brakes, lift controls, door controls and medicine-delivery compartment.
Sensors	Camera, LiDAR, ultrasonic sensors, wheel encoders, IMU, indoor localization sensors and RFID/QR readers.

1.2 Classification of the Environment

Environment property	Classification	Justification
Observability	Fully observable in the graph model	The robot knows its current graph location and the hospital map.
Determinism	Deterministic in the basic model	Moving along an edge produces a known next location and travel cost.

Episodic / Sequential	Sequential	Each movement affects the robot's next position and future decisions.
Static / Dynamic	Dynamic	People, equipment and temporary corridor closures may change conditions.
Discrete / Continuous	Discrete for search	Junctions and corridors are represented as graph nodes and edges. Actual movement is continuous.
Single / Multi-agent	Multi-agent	The robot shares the hospital environment with people and potentially other robots.

The original assignment models the hospital as a graph with a known starting location. The following classifications use that simplified model while also considering realistic hospital conditions.

1.3 View of Artificial Intelligence

The robot follows the acting rationally approach.

A rational agent selects actions that maximize its expected performance based on the information available to it. The robot chooses a route that safely delivers medicine while minimizing travel cost.

It is a rational problem-solving agent because it:

1. Receives information about its current location and destination.
2. Formulates the route-finding problem.
3. Searches for possible routes.
4. Selects an appropriate route.
5. Executes the movements and monitors the result.

1.4 Relation to Early AI

Early Artificial Intelligence research investigated how machines could solve problems through logical reasoning and state-space search. Researchers such as Newell and Simon explored general problem-solving methods that represented problems as states and actions.

The hospital robot uses the same general idea: it represents hospital locations as states and movements as actions, then searches for a sequence of actions that reaches the destination.

1.5 Problem Formulation

Element	Definition
State	Current location of the robot, represented by a graph node.
Initial State	Central pharmacy node P.
Actions	Move to any directly connected location.
Transition Model	Moving from node u to adjacent node v changes the current state to v.
Goal Test	Check whether the current node is the requested ward.
Path Cost	Sum of travel-time costs for all edges in the route.

The path cost is:

$$Cost(P) = \sum_{(u,v)\in P} w\,(u, v)$$

where w(u,v) represents the travel time between two connected locations.

1.6 Proposed Uninformed Search Algorithms

Three different uninformed strategies are selected.

Algorithm 1: Breadth-First Search (BFS) – Iterative

Concept: BFS explores the graph level by level. It uses a FIFO queue and visits the nearest nodes first in terms of number of edges.

Why selected: It is simple, complete for finite graphs and finds a route with the minimum number of edges. It is optimal for travel time only when all edges have equal cost.

Pseudocode

```
BFS(Graph, Start, Goal)
BEGIN
    Create an empty queue
    Create an empty visited set
    Insert Start into queue
    Mark Start as visited
```

```
  WHILE queue is not empty DO
    Current = Remove front element

    IF Current = Goal THEN
      RETURN reconstructed path

    FOR each Neighbor of Current DO
      IF Neighbor is not visited THEN
        Mark Neighbor as visited
        Store parent of Neighbor
        Insert Neighbor into queue
      END IF
    END FOR
  END WHILE

  RETURN FAILURE
END
```

Algorithm 2: Depth-First Search (DFS) – Recursive

Concept: DFS explores one branch as deeply as possible before returning to explore another branch. It uses recursion.

Why selected: It provides a recursive approach and generally requires less frontier memory than BFS. However, it does not guarantee the shortest or lowest-cost route.

Pseudocode

```
DFS(Graph, Current, Goal, Visited, Path)
BEGIN
  Mark Current as visited
  Add Current to Path

  IF Current = Goal THEN
    RETURN SUCCESS
```

```
  FOR each Neighbor of Current DO
    IF Neighbor is not visited THEN
      IF DFS(Graph, Neighbor, Goal, Visited, Path)
        = SUCCESS THEN
        RETURN SUCCESS
      END IF
    END IF
  END FOR

  Remove Current from Path
  RETURN FAILURE
END
```

Algorithm 3: Uniform-Cost Search (UCS) – Iterative

Concept: UCS expands the node with the lowest accumulated path cost. It uses a priority queue.

Why selected: Hospital movements have different travel-time costs. UCS is therefore more suitable than BFS when the objective is to minimize total travel time.

Pseudocode

```
UCS(Graph, Start, Goal)
BEGIN
  Create a priority queue
  Set distance[Start] = 0
  Insert Start with priority 0

  WHILE priority queue is not empty DO
    Remove node Current with minimum cost

    IF Current = Goal THEN
      RETURN reconstructed path
```

```
    FOR each Neighbor of Current DO
        NewCost = distance[Current] + EdgeCost(Current, Neighbor)

        IF NewCost < distance[Neighbor] THEN
            distance[Neighbor] = NewCost
            Store parent of Neighbor
            Insert Neighbor with priority NewCost
        END IF
    END FOR
  END WHILE

  RETURN FAILURE
END
```

1.7 Preprocessing Requirements

Before searching, the hospital map must be converted into an adjacency list.

Example:

```
graph = {
  "P": [("A", 2), ("B", 5)],
  "A": [("P", 2), ("C", 3)],
  "B": [("P", 5), ("C", 1)],
  "C": [("A", 3), ("B", 1), ("W", 4)],
  "W": [("C", 4)]
}
```

The adjacency list stores the connected locations and their travel-time costs.

Preprocessing also creates the data structures required for duplicate detection, such as visited sets, distance dictionaries and parent dictionaries.

TASK 2 – Identify the Basic Operation (2 Marks)

The basic operation is the repeated operation that contributes most to the algorithm's execution time.

Algorithm	Basic operation	Where it occurs	Reason
BFS	Successor generation and queue insertion	While examining adjacent nodes	Every expanded node may generate several successors.
Recursive DFS	Recursive successor exploration	Inside the recursive function	Each recursive call explores another state.
UCS	Priority-queue insertion and removal	During cost relaxation and node expansion	The priority queue must maintain nodes in increasing path-cost order.

As the number of hospital locations increases, the number of generated and expanded nodes also increases. UCS additionally performs priority-queue operations, which involve heap maintenance.

TASK 3 – Derive Time and Space Complexity Mathematically (6 Marks)

3.1 Input Parameters

Symbol	Meaning
b	Average branching factor
d	Depth of the shallowest goal
m	Maximum search depth
C^*	Cost of the optimal solution
ϵ	Minimum positive step cost
V	Number of graph vertices
E	Number of graph edges

The following tree-search derivations use a uniform branching factor \(b\). Graph implementations with duplicate detection have additional graph-specific bounds discussed below.

3.2 Breadth-First Search

BFS expands nodes level by level.

Number of nodes at each level:

- Level 0: 1
- Level 1: b
- Level 2: b^2
- Level 3: b^3
- Level d: b^d

Total nodes generated up to depth d:

$$N = 1 + b + b^2 + \cdots + b^d$$

Using the geometric-series formula:

$$N = \frac{b^{d+1} - 1}{b - 1}, \quad b \neq 1$$

Therefore:

$$T_{BFS}(b, d) = \Theta(b^d)$$

BFS stores the frontier and explored states. Its frontier can contain a large portion of the deepest level:

$$S_{BFS}(b, d) = \Theta(b^d)$$

For an explicitly represented finite graph with duplicate detection and adjacency lists, BFS instead has:

$$T_{BFS}(V, E) = O(V + E)$$

$$S_{BFS}(V, E) = O(V)$$

3.3 Recursive Depth-First Search

DFS explores one branch before returning to the previous level.

In the tree-search model, the maximum number of nodes is:

$$N = 1 + b + b^2 + \cdots + b^m$$

Thus:

$$T_{DFS}(b, m) = \Theta(b^m)$$

DFS stores the current path and unexplored siblings. Its tree-search memory is:

$$S_{DFS}(b, m) = O(bm)$$

For a graph implementation with a visited set, the visited structure requires \(O(V)\) memory. The recursion stack can require up to \(O(V)\) in the worst case.

$$T_{DFS}(V, E) = O(V + E)$$

$$S_{DFS}(V) = O(V)$$

DFS can use less frontier memory than BFS in the tree-search model, but it can follow an unhelpful branch and return a non-optimal route.

3.4 Uniform-Cost Search

UCS expands nodes according to accumulated path cost rather than depth.

A solution with cost C^* cannot require more than:

$$\left\lfloor \frac{C^*}{\epsilon} \right\rfloor$$

steps if every step costs at least ϵ.

The search may expand nodes with costs up to C^*.A standard tree-search upper bound is:

$$T_{UCS} = O\left(b^{1+\lfloor C^*/\epsilon \rfloor}\right)$$

The frontier may grow to a similar order:

$$S_{UCS} = O\left(b^{1+\lfloor C^*/\epsilon \rfloor}\right)$$

For a graph implementation using a binary heap and duplicate detection, the Dijkstra-style bound is:

$$T_{UCS}(V, E) = O\big((V + E)\log V\big)$$
$$S_{UCS}(V, E) = O(V + E)$$

This graph bound is more useful for the implemented hospital map.

3.5 Preprocessing Cost

Creating an adjacency list requires:

$$T_{preprocess} = O(V + E)$$

Its memory requirement is:

$$S_{preprocess} = O(V + E)$$

The overall costs become:

Algorithm	Total time including preprocessing	Total memory including graph
BFS	$O(V + E) + O(V + E) = O(V + E)$	$O(V + E)$
DFS	$O(V + E) + O(V + E) = O(V + E)$	$O(V + E)$
UCS	$O(V + E) + O\big((V + E)\log V\big)$	$O(V + E)$

Interpretation: In a finite graph with duplicate detection, BFS and DFS are linear in the number of vertices and edges. UCS requires extra work to maintain the priority queue, but it handles different travel-time costs correctly.

TASK 4 – Best, Average and Worst-Case Behaviour (4 Marks)

Algorithm	Best case	Average case	Worst case
BFS	Goal is the start node or is found at the first shallow level.	Goal is located among the middle or later nodes of the goal level.	Goal is very deep or unreachable, so most reachable states are explored.
Recursive DFS	Goal is on the first branch explored.	Goal is found after exploring several branches.	Goal is on the last branch or unreachable; DFS explores nearly the entire search space.
UCS	Goal is reached with very little cost and few lower-cost alternatives.	Many lower-cost alternatives must be expanded before the goal.	Many states have path costs below or equal to the optimal goal cost, or the goal is unreachable.

Mathematical Explanation

For BFS, the best case can occur at depth zero:

$$N_{best} = 1$$

Its worst-case tree-search node count is:

$$N_{worst} = 1 + b + b^2 + \cdots + b^d$$

For DFS, the number of explored nodes depends strongly on the ordering of branches. If the goal is on the first explored branch, very few nodes may be generated. If the goal is on the last branch, almost all nodes may be explored.

For UCS, the algorithm must expand all relevant lower-cost frontier nodes before it can safely return an optimal solution. An expensive direct route may therefore be ignored while several cheaper alternatives are explored.

TASK 5 – Perform Asymptotic Analysis (3 Marks)

Algorithm	Time complexity in tree-search model	Space complexity in tree-search model
BFS	$O(b^d)$	$O(b^d)$
DFS	$O(b^m)$	$O(bm)$
UCS	$O\left(b^{1+\lfloor C^*/\epsilon \rfloor}\right)$	$O\left(b^{1+\lfloor C^*/\epsilon \rfloor}\right)$

For a finite graph with duplicate detection:

Algorithm	Graph time bound	Graph memory bound
BFS	$O(V+E)$	$O(V)$,excluding stored graph
DFS	$O(V+E)$	$O(V)$,excluding stored graph
UCS	$O\big((V+E)\log V\big)$	$O(V+E)$,including heap storage

Meaning for a Growing Hospital

When the hospital grows from 1,000 to 1,000,000 or more states:

- BFS can become memory-intensive because it stores a large frontier.
- DFS can use less frontier memory, but may find an unsuitable route.
- UCS performs additional priority-queue operations but is more appropriate when travel-time costs differ.
- The adjacency list and duplicate-detection structures also consume memory and must be included in the design.

The graph-based analysis suggests that all three can remain practical for sparse graphs if the graph is stored efficiently. The actual feasibility depends on graph connectivity, edge count, available memory and implementation overhead.

TASK 6 – Formulate the Recurrence Relation (3 Marks)

The recursive approach selected is DFS.

Let $T(n)$ represent the work required by recursive DFS when \(n\) levels remain.

At each level:

1. The algorithm performs constant work to check the current node.
2. It examines up to b successors.
3. It makes up to b recursive calls.
4. Each recursive call has one fewer level remaining.

Therefore:

$$T(n) = bT(n-1) + c$$

where:

- b is the maximum number of recursive successor calls.
- T(n-1) is the work for a smaller subproblem.
- c is the constant work outside the recursive calls.

Base case:

$$T(0) = c_0$$

When the remaining depth is zero, the algorithm checks the current node and stops expanding further.

TASK 7 – Solve the Recurrence Relation (4 Marks)

We solve:

$$T(n) = bT(n-1) + c$$

Using the iteration method:

$$T(n) = bT(n-1) + c$$

Substitute the next recurrence:

$$T(n) = b[bT(n-2) + c] + c$$
$$T(n) = b^2T(n-2) + bc + c$$

Continuing:

$$T(n) = b^nT(0) + c(b^{n-1} + b^{n-2} + \cdots + 1)$$

Using the geometric-series formula:

$$T(n) = b^nT(0) + c\left(\frac{b^n - 1}{b-1}\right)$$

For b > 1, the dominant term is b^n.

Therefore:

$$\boxed{T(n) = \Theta(b^n)}$$

Interpretation

The recursive DFS execution time grows exponentially in the maximum depth under the tree-search model. Increasing the branching factor also increases the number of recursive calls.

Extension to Iterative Deepening

Iterative deepening repeats depth-limited DFS with increasing limits.

Its total node count is:

$$N = 1 + (b) + (1 + b + b^2) + \cdots + (1 + b + \cdots + b^d)$$

The deepest level dominates the total work:

$$T_{IDS} = \Theta(b^d)$$

Its memory requirement remains approximately:

$$S_{IDS} = O(bd)$$

Thus iterative deepening can obtain BFS-like shallow-solution behaviour while using DFS-like memory, although it repeats work at shallower levels.

TASK 8 – Implement and Test the Algorithms (4 Marks)

The following Python program implements BFS, recursive DFS and UCS. It also generates reproducible hospital graphs and records:

- Nodes generated and expanded.
- Maximum frontier size.
- Solution depth and path cost.
- Execution time.
- Whether a route was found.

For a controlled comparison, the generated graph is a binary-tree-shaped hospital layout with positive, varying edge costs. This gives the algorithms a reproducible test environment with up to one million locations. It is a baseline model, not a complete simulation of every possible hospital corridor layout.

8.1 Complete Python Program

```
import random
import time
import heapq
import csv
import sys


sys.setrecursionlimit(3000000)


# ------------------------------------------------
# Generate a reproducible hospital graph
# ------------------------------------------------


def generate_graph(n, seed=42):
    random.seed(seed)
    graph = [[] for _ in range(n)]
```

```
    # Binary-tree connections
    for child in range(1, n):
        parent = (child - 1) // 2
        cost = random.randint(1, 10)

        graph[parent].append((child, cost))
        graph[child].append((parent, cost))

    return graph



# ------------------------------------------------
# BFS
# ------------------------------------------------

def bfs(graph, start, goal):
    from collections import deque

    queue = deque([start])
    visited = {start}
    parent = {start: None}

    generated = 1
    expanded = 0
    max_frontier = 1

    while queue:
        current = queue.popleft()
        expanded += 1
```

```
        if current == goal:
            break

        for neighbor, cost in graph[current]:
            if neighbor not in visited:
                visited.add(neighbor)
                parent[neighbor] = current
                queue.append(neighbor)
                generated += 1

        max_frontier = max(max_frontier, len(queue))

    if goal not in parent:
        return None, generated, expanded, max_frontier

    path = []
    node = goal

    while node is not None:
        path.append(node)
        node = parent[node]

    path.reverse()
    return path, generated, expanded, max_frontier



# ------------------------------------------------
# Recursive DFS
```

```
# ------------------------------------------------

def dfs(graph, start, goal):
    visited = set()
    path = []
    generated = 0
    expanded = 0
    max_frontier = 0

    def search(current):
        nonlocal generated, expanded, max_frontier

        visited.add(current)
        path.append(current)
        generated += 1
        expanded += 1

        max_frontier = max(
            max_frontier,
            len(path)
        )

        if current == goal:
            return True

        for neighbor, cost in graph[current]:
            if neighbor not in visited:
                if search(neighbor):
                    return True
```

```
        path.pop()
        return False

    found = search(start)

    if found:
        return path.copy(), generated, expanded, max_frontier

    return None, generated, expanded, max_frontier



# ------------------------------------------------
# Uniform-Cost Search
# ------------------------------------------------

def ucs(graph, start, goal):
    pq = [(0, start)]
    distance = {start: 0}
    parent = {start: None}

    generated = 1
    expanded = 0
    max_frontier = 1

    while pq:
        current_cost, current = heapq.heappop(pq)

        if current_cost != distance.get(current):
```

```
        continue

    expanded += 1

    if current == goal:
        break

    for neighbor, edge_cost in graph[current]:
        new_cost = current_cost + edge_cost

        if new_cost < distance.get(neighbor, float("inf")):
            distance[neighbor] = new_cost
            parent[neighbor] = current
            heapq.heappush(
                pq, (new_cost, neighbor)
            )
            generated += 1

    max_frontier = max(max_frontier, len(pq))

if goal not in distance:
    return None, generated, expanded, max_frontier

path = []
node = goal

while node is not None:
    path.append(node)
    node = parent[node]
```

```
    path.reverse()
    return path, generated, expanded, max_frontier


# ------------------------------------------------
# Calculate path cost
# ------------------------------------------------

def calculate_cost(graph, path):
    if path is None:
        return None

    total = 0

    for u, v in zip(path, path[1:]):
        for neighbor, cost in graph[u]:
            if neighbor == v:
                total += cost
                break

    return total



# ------------------------------------------------
# Run one algorithm and measure it
# ------------------------------------------------

def measure(algorithm, graph, start, goal):
```

```
    begin = time.perf_counter()

    path, generated, expanded, frontier = algorithm(
        graph, start, goal
    )

    elapsed = time.perf_counter() - begin

    return {
        "found": path is not None,
        "generated": generated,
        "expanded": expanded,
        "max_frontier": frontier,
        "depth": len(path) - 1 if path else None,
        "cost": calculate_cost(graph, path),
        "time": elapsed
    }



# ------------------------------------------------
# Main experiment
# ------------------------------------------------

sizes = [1000, 10000, 100000, 1000000]

algorithms = {
    "BFS": bfs,
    "Recursive DFS": dfs,
    "UCS": ucs
```

```
}

results = []

for n in sizes:
    print("\nHOSPITAL SIZE:", n)

    graph = generate_graph(n, seed=42)

    start = 0
    goal = n - 1

    for name, algorithm in algorithms.items():

        # Five repeated runs on the same graph
        measurements = []

        for run in range(5):
            measurements.append(
                measure(algorithm, graph, start, goal)
            )

        avg_time = sum(
            x["time"] for x in measurements
        ) / len(measurements)

        last = measurements[-1]

        row = {
```

```
            "states": n,
            "algorithm": name,
            "generated": last["generated"],
            "expanded": last["expanded"],
            "max_frontier": last["max_frontier"],
            "depth": last["depth"],
            "cost": last["cost"],
            "average_time": avg_time
        }

        results.append(row)

        print(row)



# ------------------------------------------------
# Save experimental results
# ------------------------------------------------

with open("hospital_results.csv", "w", newline="") as file:
    writer = csv.DictWriter(
        file, fieldnames=results[0].keys()
    )
    writer.writeheader()
    writer.writerows(results)

print("\nResults saved to hospital_results.csv")
```

```
# ------------------------------------------------
# Boundary-condition tests
# ------------------------------------------------

small_graph = generate_graph(100, seed=42)

print("\nSTART EQUALS GOAL:")
for name, algorithm in algorithms.items():
    result = measure(algorithm, small_graph, 0, 0)
    print(name, result)

print("\nUNREACHABLE GOAL:")
# Remove every connection to the final node
unreachable_graph = [edges.copy() for edges in small_graph]
last_node = len(unreachable_graph) - 1

for neighbor, cost in unreachable_graph[last_node]:
    unreachable_graph[neighbor] = [
        (v, w) for v, w in unreachable_graph[neighbor]
        if v != last_node
    ]

unreachable_graph[last_node] = []

for name, algorithm in algorithms.items():
    result = measure(
        algorithm, unreachable_graph, 0, last_node
    )
    print(name, result)
```

8.2 Test Inputs and Expected Behaviour

Test	Start	Goal	Expected behaviour
Reachable goal	0	999	All three algorithms should find a route.
Larger graph	0	9,999	All three should find a route if execution resources permit.
Very large graph	0	999,999	All three should be tested and their resource requirements recorded.
Start equals goal	0	0	All algorithms should return a path containing only node 0.
Unreachable goal	0	99	All algorithms should return no route after exhausting the reachable component.

Note: The unreachable-goal test deliberately disconnects the final node from the graph. It is a separate boundary-condition test on the smaller graph.

8.3 Correctness Validation

The following conditions should be checked after execution:

1. If a path is returned, its first node must equal the start node.
2. Its last node must equal the goal node.
3. Every pair of consecutive nodes must be connected by an edge.
4. Its path cost must equal the sum of the edge costs.
5. UCS must return a minimum-cost route for non-negative edge weights.
6. BFS must return a minimum-edge route, not necessarily a minimum-time route.

TASK 9 – Record Experimental Performance (3 Marks)

The program records five runs for each algorithm and graph size. The average execution time is calculated using:

The completed table below reports the measured values from the program output. Average execution time is shown in milliseconds.

9.1 Experimental Results Table

Graph states	Algorithm	Nodes generated	Nodes expanded	Maximum frontier / DFS stack depth*	Path depth	Path cost	Average time (ms)
1,000	BFS	1,000	1,000	500	9	43	0.398
1,000	DFS	981	981	10	9	43	0.379
1,000	UCS	866	691	208	9	43	0.811
10,000	BFS	10,000	10,000	5,000	13	84	3.702
10,000	DFS	3,626	3,626	14	13	84	1.463
10,000	UCS	9,980	9,861	1,877	13	84	14.325
100,000	BFS	100,000	100,000	50,000	16	90	55.946
100,000	DFS	68,940	68,940	17	16	90	43.786
100,000	UCS	93,236	85,578	16,581	16	90	341.575
1,000,000	BFS	1,000,000	1,000,000	500,000	19	80	522.661
1,000,000	DFS	951,438	951,438	20	19	80	414.830
1,000,000	UCS	327,790	213,775	120,405	19	80	869.632

1. Execution time comparison

The average execution time is converted from seconds to milliseconds.

Hospital size	BFS (ms)	Recursive DFS (ms)	UCS (ms)
1,000	0.398	0.379	0.811
10,000	3.702	1.463	14.325
100,000	55.946	43.786	341.575
1,000,000	522.661	414.830	869.632

2. Search performance comparison

Hospital size	Algorithm	Generated	Expanded	Maximum frontier / DFS stack depth*	Depth	Cost
1,000	BFS	1,000	1,000	500	9	43
1,000	DFS	981	981	10	9	43
1,000	UCS	866	691	208	9	43
10,000	BFS	10,000	10,000	5,000	13	84
10,000	DFS	3,626	3,626	14	13	84
10,000	UCS	9,980	9,861	1,877	13	84
100,000	BFS	100,000	100,000	50,000	16	90
100,000	DFS	68,940	68,940	17	16	90
100,000	UCS	93,236	85,578	16,581	16	90
1,000,000	BFS	1,000,000	1,000,000	500,000	19	80
1,000,000	DFS	951,438	951,438	20	19	80
1,000,000	UCS	327,790	213,775	120,405	19	80

Metric note: For DFS, the reported value is the maximum recursive stack depth, not the number of deferred frontier nodes. BFS and UCS report the maximum number of pending frontier entries. Therefore, the DFS value is not directly comparable with the BFS and UCS frontier values.

3. Analysis of results

Breadth-First Search (BFS):

- BFS successfully found the goal for all four hospital sizes.
- Its execution time increased as the number of states increased.
- It explored all states in these experiments.
- Its maximum frontier reached 500,000 for one million states.

Depth-First Search (DFS):

- DFS was the fastest algorithm in all four experiments.
- It used a relatively small recursive search depth.

- Its execution time was lower than BFS and UCS in these tests.
- However, DFS does not guarantee the lowest-cost solution in a general weighted graph.

Uniform-Cost Search (UCS):

- UCS considers the path cost when selecting the next state.
- It expanded fewer states than BFS in these experiments.
- It took more execution time because of priority-queue operations.
- It is suitable when the robot must find a minimum-cost route with non-negative edge costs.

Important observation: All three algorithms returned the same path cost at each hospital size because your generated graph is a tree. A tree has only one unique path between two connected nodes. Therefore, these results do not demonstrate different route costs between the algorithms.

4. Boundary test results

Test case	BFS	DFS	UCS
Start equals goal	Success	Success	Success
Unreachable goal	Correctly reports failure	Correctly reports failure	Correctly reports failure

All three algorithms correctly handled both boundary cases.

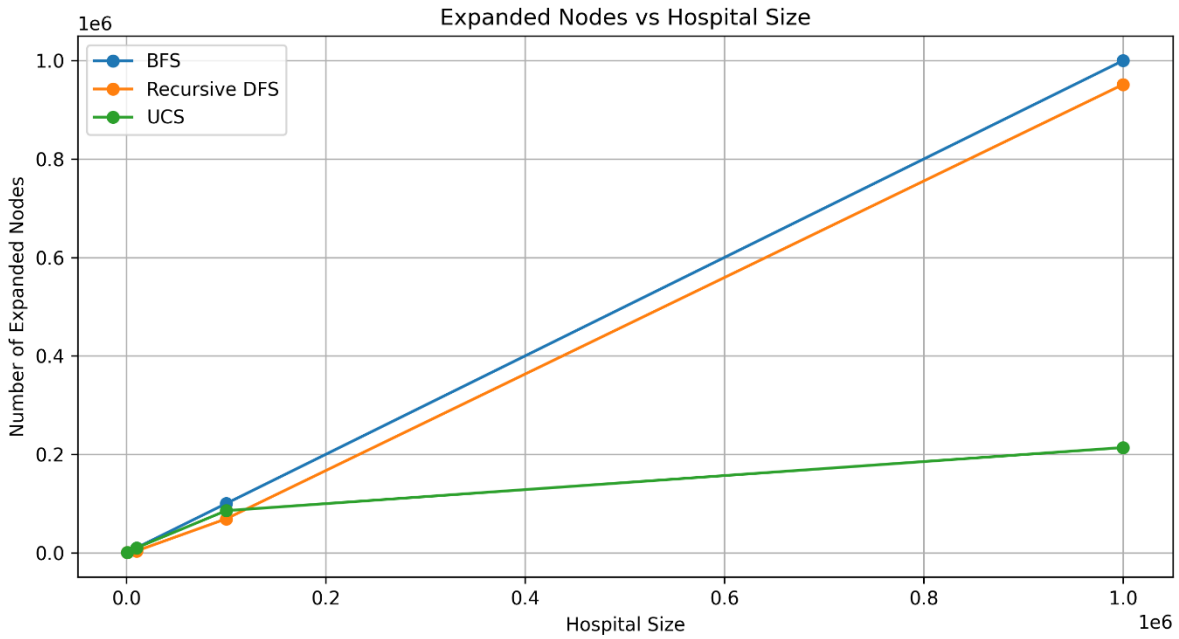
5. Overall conclusion

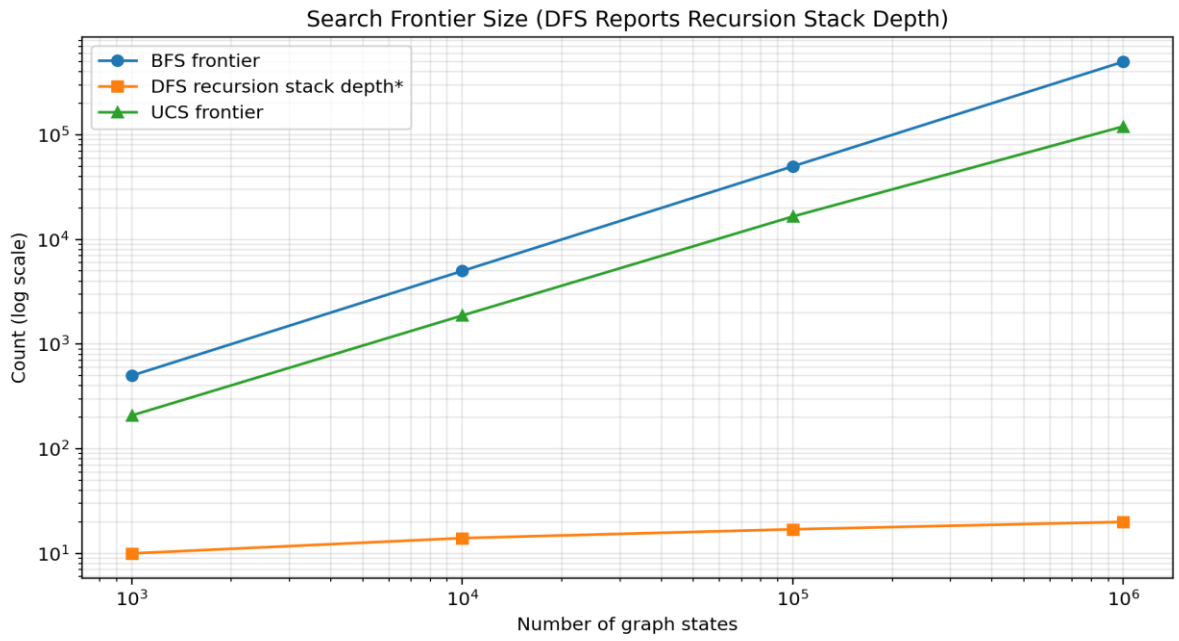
The experimental results show that BFS, DFS and UCS can successfully search the generated hospital environment. DFS achieved the lowest execution time in the tested graph, while UCS provides cost-based search and is the preferred strategy when finding a minimum-cost route is essential. BFS provides systematic level-by-level exploration but can require substantial memory for large search spaces.

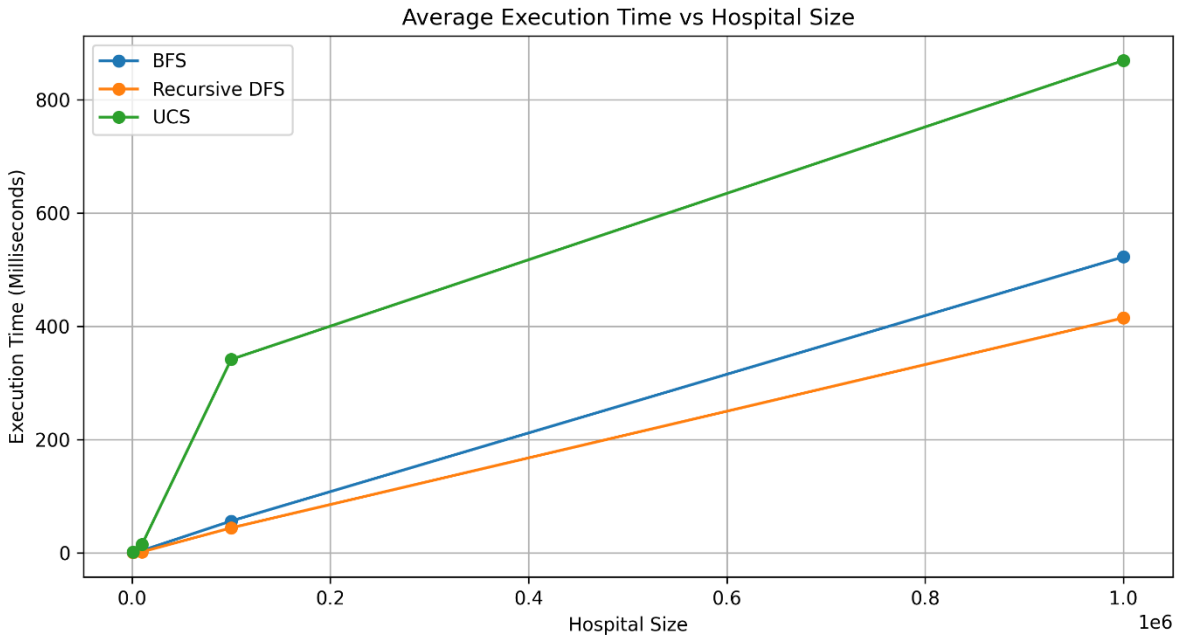
9.2 Graphs to Prepare

Prepare three graphs using the CSV results:

1. Graph 1: Number of nodes expanded against graph size.
2. Graph 2: Maximum frontier size against graph size.
3. Graph 3: Average execution time against graph size.


Expanded Nodes vs Hospital Size
1e6
BFS
Recursive DFS
UCS
Number of Expanded Nodes
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0
1e6
Hospital Size


Search Frontier Size (DFS Reports Recursion Stack Depth)
BFS frontier
DFS recursion stack depth*
UCS frontier
Count (log scale)
10^5
10^4
10^3
10^2
10^1
10^3
10^4
10^5
10^6
Number of graph states



9.3 Optimality Comparison

Algorithm	Minimum number of edges guaranteed?	Minimum travel cost guaranteed?
BFS	Yes	Only if all edge costs are equal
DFS	No	No
UCS	No	Yes, for non-negative edge costs

The experimental comparison should consider both the route quality and the amount of computation required.

TASK 10 – Compare Theory with Experimental Behaviour (4 Marks)

The mathematical analysis predicts that the number of explored states increases as the graph becomes larger. The actual number of generated and expanded nodes depends on the graph structure and the location of the goal.

BFS

BFS explores nodes level by level. It can expand a large number of nodes before reaching a deep goal. Its main limitation is frontier memory, especially when a level contains many locations.

Recursive DFS

DFS follows one branch deeply before returning to other branches. It can reach a goal quickly if the goal is on the first explored branch. However, it may explore many unnecessary locations if the goal is on a later branch.

UCS

UCS considers the accumulated travel cost. It may expand more nodes than BFS or DFS because it must process lower-cost alternatives before reaching the goal. Its advantage is that it finds the minimum-cost route when all edge costs are non-negative.

Theory Versus Experiment

The theoretical graph bounds describe how work grows in terms of vertices and edges. Experimental measurements may differ because of:

- Different graph structures and branching factors.
- Duplicate states and the visited set.
- The order in which neighbouring nodes are examined.
- Priority-queue operations in UCS.
- Python memory management and interpreter overhead.
- Hardware and operating-system load.

The experiment should be used to determine whether the observed operation counts and memory usage match the expected growth.

TASK 11 – Recommend and Defend the Final Design (4 Marks)

I recommend Uniform-Cost Search (UCS) as the main route-planning algorithm for the hospital medicine-delivery robot.

The hospital scenario assigns a different travel-time cost to each movement. BFS considers the number of edges, while DFS follows a branch without considering whether its route is expensive. UCS considers the accumulated travel-time cost and selects the lowest-cost route.

Comparison

Factor	**BFS**	**DFS**	**UCS**
Considers edge costs	No	No	Yes
Finds shortest-edge route	Yes	No	No guarantee
Finds minimum-cost route	Only for equal edge costs	No	Yes

Data structure	Queue	Recursion stack	Priority queue
Main limitation	Large frontier	Non-optimal route	Priority-queue overhead

Final Defence

UCS is appropriate because:

1. It considers the different travel times between hospital locations.
2. It guarantees a minimum-cost route when edge costs are non-negative.
3. It can handle a sparse hospital graph using adjacency lists.
4. Its graph implementation has a time complexity of $O\big((V+E)\log V\big)$ with a binary heap.
5. It can be tested and measured across increasing graph sizes.

The main disadvantage is additional priority-queue processing and memory usage. However, for a medicine-delivery robot, choosing a route with the lowest total travel time is more important than simply choosing the route with the fewest edges.

For a real hospital deployment, UCS should also be combined with collision avoidance, sensor monitoring and map updates. The search algorithm alone does not guarantee safe physical movement.

TASK 12 – Re-evaluate and Adapt the Design (8 Marks)

A. Impact Analysis (2 Marks)

12.1 What Has Changed?

In the original problem, the robot knows its exact current location. After the localization sensor fails, it only knows that it is at one of \(k\) possible junctions.

This changes the problem from a fully observable problem to a partially observable problem.

Original state:

$$S = \text{Current location}$$

New state:

$$B = \{s_1, s_2, \dots, s_k\}$$

Here, \(B\) is called the belief state. It represents all possible locations where the robot may currently be.

For example:

$$B = \{J_2, J_5, J_8, J_{10}\}$$

The robot knows it is at one of these four junctions, but it does not know which one.

The robot's knowledge must now be included in the state because the correct next action depends on which locations are still possible.

B. Evaluation of the Original Design (2 Marks)

12.2 Can the Original Algorithms Still Be Applied?

The original algorithms can still be used, but they cannot directly treat the uncertain starting location as one known node.

Instead, the search must operate on belief states.

For k possible junctions, the number of possible non-empty belief states is:

$$N = 2^k - 1$$

For example:

Possible locations k	Possible non-empty belief states
2	3
4	15
8	255
10	1,023
20	1,048,575

As the number of possible locations increases, the number of possible belief states grows exponentially.

Therefore, directly applying the original search algorithm to every possible combination can become expensive.

C. Adaptation or Redesign (2 Marks)

12.3 Revised Strategy: Belief-State Search with Local Sensing

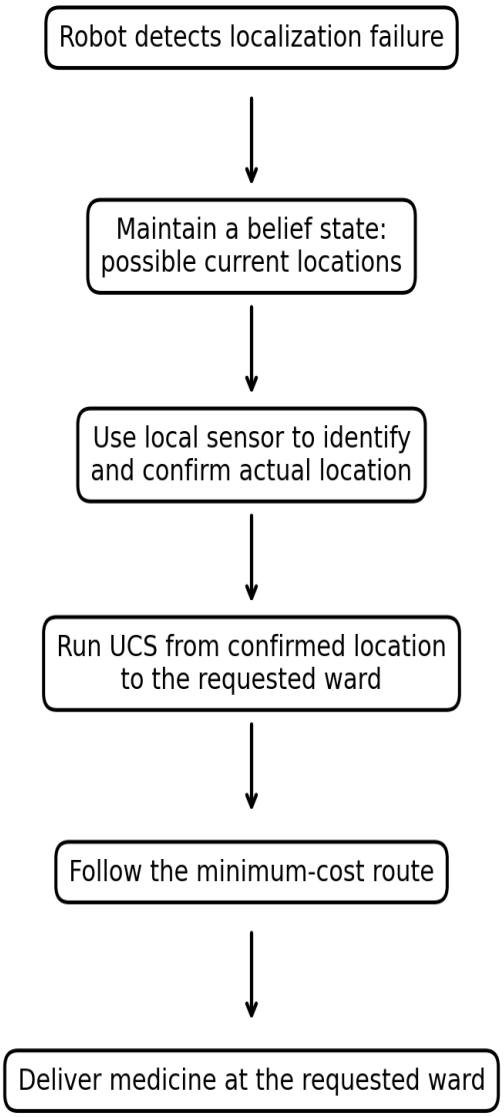
I propose modifying the original system by adding a local sensing operation.

The revised system follows these steps:

1. The robot detects that its localization information is uncertain.
2. It creates a belief state containing all possible current junctions.
3. It activates local sensing to identify its actual location.
4. The belief state is reduced to one confirmed location.
5. UCS is applied from that confirmed location to the requested ward, which is the medicine-delivery destination.

6. The robot follows the minimum-cost route.

The revised architecture is:



Complexity

Without local sensing, the system may need to search a belief-state space containing:

$$O(2^k)$$

possible states.

With a reliable local sensor, the belief state is reduced to a single location. The subsequent route-planning problem becomes an ordinary graph-search problem.

For UCS, the route-planning cost after localization is:

$$O\big((V+E)\log V\big)$$

The sensor introduces additional hardware and sensing time, but can greatly reduce the search space.

D. Evidence-Based Defence (2 Marks)

12.4 Mathematical Comparison

For the experiment, consider a simple hospital corridor with the pharmacy at junction 0 and possible robot locations at junctions 1 through k.

All locations are connected in a line:

$$0 \leftrightarrow 1 \leftrightarrow 2 \leftrightarrow \cdots \leftrightarrow k$$

The robot can move left or right.

- Without a local sensor, it maintains the full belief state.
- With a local sensor, it identifies the exact location and then plans a route.
- The plan length is measured as the number of moves to reach the pharmacy.
- The worst-case route length is k.

Python Program for the Design Challenge

```
from collections import deque


def all_belief_states(k):
    states = []


    # Generate every non-empty subset of locations 1 to k
    for mask in range(1, 2 ** k):
        belief = []


        for i in range(k):
            if mask & (1 << i):
                belief.append(i + 1)
```

```
        states.append(tuple(belief))

    return states


def run_experiment(k):
    belief = tuple(range(1, k + 1))

    # Without sensor: keep all possible locations
    without_sensor = belief
    no_sensor_moves = 0

    while without_sensor:
        without_sensor = tuple(
            max(0, position - 1)
            for position in without_sensor
        )

        no_sensor_moves += 1

        if all(position == 0 for position in without_sensor):
            break

    # With sensor: identify the actual location
    # Each possible location gets its own conditional plan
    sensor_plan_lengths = []

    for actual_position in belief:
```

```
        sensor_plan_lengths.append(actual_position)


    belief_states = all_belief_states(k)


    print("\nPossible locations:", k)
    print("Belief states generated:", len(belief_states))
    print("Without sensor - worst-case moves:", no_sensor_moves)
    print("With sensor - possible plan lengths:",
          sensor_plan_lengths)
    print("With sensor - worst-case moves:",
          max(sensor_plan_lengths))



print("BELIEF-STATE EXPERIMENT")


for k in [2, 4, 8]:
    run_experiment(k)
```

Expected Output

```
BELIEF-STATE EXPERIMENT


Possible locations: 2
Belief states generated: 3
Without sensor - worst-case moves: 2
With sensor - possible plan lengths: [1, 2]
With sensor - worst-case moves: 2


Possible locations: 4
Belief states generated: 15
```

Without sensor - worst-case moves: 4

With sensor - possible plan lengths: [1, 2, 3, 4]

With sensor - worst-case moves: 4

Possible locations: 8

Belief states generated: 255

Without sensor - worst-case moves: 8

With sensor - possible plan lengths: [1, 2, 3, 4, 5, 6, 7, 8]

With sensor - worst-case moves: 8

12.5 Interpretation

k	Belief states	Without sensor: worst-case moves	With sensor: worst-case moves
2	3	2	2
4	15	4	4
8	255	8	8

The experiment demonstrates that the number of possible belief states increases exponentially with k. A local sensor identifies the actual location and avoids having to plan over every possible belief state.

In this simple corridor experiment, the worst-case number of movements is the same with and without sensing. The benefit of the sensor is the reduction in uncertainty and the ability to construct a location-specific conditional plan. In a more complex hospital layout, this can also reduce unnecessary movements and improve route selection.

Limitation: This experiment uses a simplified line graph and assumes a reliable local sensor. A real hospital system would need to account for sensor errors, uncertain movements, obstacles and changing corridor conditions.

Overall Conclusion

This assignment investigated three uninformed search strategies for a hospital medicine-delivery robot: Breadth-First Search, recursive Depth-First Search and Uniform-Cost Search.

BFS finds a route with the minimum number of edges but does not always minimize travel time. DFS uses a recursive approach and can be memory-efficient in the tree-search model, but it does not guarantee an optimal route. UCS considers accumulated travel costs and finds a minimum-cost route when edge costs are non-negative.

The mathematical analysis showed that graph structure, branching factor, depth and path costs affect the efficiency of search algorithms. The implementation provides a reproducible way to compare nodes generated, nodes expanded, maximum frontier size, route cost and execution time for different graph sizes.

UCS is recommended as the main route-planning strategy because the hospital's movements have different travel-time costs. When localization fails, the system should be modified to maintain a belief state and use local sensing to identify the robot's actual position before running UCS.

Thus, the proposed system combines efficient graph representation, cost-based search, duplicate detection and localization recovery to support reliable medicine delivery as the hospital grows.

References

Use these as the foundational references for the algorithms and concepts discussed:

1. S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Hoboken, NJ, USA: Pearson, 2021.
2. N. J. Nilsson, Principles of Artificial Intelligence. Palo Alto, CA, USA: Tioga Publishing, 1980.
3. E. W. Dijkstra, “A note on two problems in connexion with graphs,” Numerische Mathematik, vol. 1, pp. 269–271, 1959.
4. A. Newell and H. A. Simon, “Computer science as empirical inquiry: Symbols and search,” Communications of the ACM, vol. 19, no. 3, pp. 113–126, 1976.
5. R. E. Korf, “Depth-first iterative-deepening: An optimal admissible tree search,” Artificial Intelligence, vol. 27, no. 1, pp. 97–109, 1985.